HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

GRADUATION RESEARCH REPORT (GR1)

Topic: Design and Development of a Web-Based Fitness Platform for Beginner Gym Users

Project Code Name: FiFB (Fitness for Beginners)

NGÔ ĐỨC HUY

Huy.nd235946@sis.hust.edu.vn

Program: Global ICT

Supervisor: Professor Cao Tuan Dung

Department: Department of Computer Science

School: School of Information and Communications Technology

HANOI, 07/2026

Table of Contents

Chapter 1 – Introduction *5*

1.1 Context 5

1.2 Problem Statement and Practical Significance 5

1.3 Contribution and Position Regarding Novelty 6

1.4 Motivation and Development Orientation 6

1.5 Objectives and Scope of GR1 7

1.6 Research and Development Method 7

1.7 Planned Roadmap from GR1 to GR2 8

Chapter 2 – Survey and Related Products *9*

2.1 Survey Method and Evaluation Criteria 9

2.2 JEFIT 9

2.3 Hevy 10

2.4 ExerciseDB as a Data Source 11

2.5 Functional Comparison 12

2.6 Frontend and User-Experience Comparison 12

2.7 Derived Product Direction and Quality Attributes 13

Chapter 3 – Theoretical Foundations and Technology Stack *14*

3.1 End-to-End Technical Workflow 14

3.2 Client–Server Architecture and RESTful Communication 14

3.3 Frontend Technology Stack 15

3.4 Authentication, JWT, and Password Protection 15

3.5 Role-Based Access Control 16

3.6 Relational Database Design and Supabase Hosting 16

3.7 ExerciseDB V1 API Data Acquisition and Processing 17

3.8 Backend Technology Stack 17

3.9 Validation, Testing, and Code-Quality Support 17

3.10 Technology Summary and Scope Boundary 18

Chapter 4 – Design and Development *19*

4.1 Product Overview and Actors 19

4.2 Functional Requirements **20**
4.2.1 Visitor Requirements 20
4.2.2 User Requirements 20
4.2.3 Trainer Requirements 21
4.2.4 Admin Requirements 21
4.3 Non-Functional Requirements **22**
4.4 System Architecture and Project Structure **23**
4.5 Database Design **24**
4.6 Backend Design and Implementation **25**
4.6.1 API Resource Groups 25
4.6.2 Authentication and User Access Management 26
4.6.3 Exercise Data Acquisition and Normalization 27
4.6.4 Exercise Content Lifecycle and Moderation 28
4.6.5 Exercise Search and Relational Data Retrieval 29
4.6.6 Validation, Transactions, and Error Handling 30
4.7 Frontend Design and Implementation **31**
4.7.1 Routing and Layout Separation 31
4.7.2 API Client and Authentication State 31
4.7.3 Search, Filters, and Exercise Details 31
4.7.4 Administrative and Trainer Interfaces 31
4.7.5 Reusable UI Components 32
4.8 Principal End-to-End Workflows **32**
Chapter 5 – Results and Evaluation ***33***
5.1 Results Achieved in GR1 ..... **33**
5.2 Exercise Data Acquisition Result **33**
5.3 System Operation **33**
5.4 Application Interfaces **34**
5.5 Qualitative Evaluation of Software Quality **37**
5.6 Limitations **38**
5.7 Development Plan for GR2 ..... **38**
References ***40***
Appendices ***42***
Appendix A – Core API Documentation **42**
Appendix B – Database Schema **42**
Appendix C – Focused Repository Structure **42**

LIST OF ABBREVIATIONS

Abbreviation	Full Term
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Delivery
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DBMS	Database Management System
ERD	Entity–Relationship Diagram
FR	Functional Requirement
GIF	Graphics Interchange Format
GR1	Graduation Research 1
GR2	Graduation Research 2
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVP	Minimum Viable Product
NFR	Non-Functional Requirement
OTP	One-Time Password
RBAC	Role-Based Access Control
REST	Representational State Transfer
RDBMS	Relational Database Management System
SPA	Single-Page Application
SQL	Structured Query Language
UI/UX	User Interface / User Experience
UUID	Universally Unique Identifier

Chapter 1 – Introduction

1.1 Context

Physical activity plays an important role in maintaining health, yet many people still do not meet recommended activity levels. According to the World Health Organization, around 31% of adults worldwide were insufficiently active in 2022 [1], [2]. Although digital platforms cannot replace medical advice or professional coaching, they can make exercise information easier to access by providing clear instructions, visual demonstrations, and structured guidance.

The growth of digital fitness services has changed how people discover exercises, track progress, and communicate with trainers. However, many existing applications are designed for experienced users, professional coaches, or commercial training programs. As a result, beginners may find them difficult to use when they only need basic information about exercise techniques, target muscles, and required equipment.

FiFB is developed as the foundation of a web-based fitness community. Rather than managing a specific gym facility, the system aims to help users access structured exercise information and gradually connect with trainers and other community members. In GR1, the project focuses on the essential technical foundation, including role-based access control, authentication, relational database design, exercise taxonomy, and core management functions. Advanced features such as artificial intelligence and automated training recommendations are reserved for future development.

1.2 Problem Statement and Practical Significance

FiFB addresses three major pain points that currently hinder fitness enthusiasts. First, exercise information is highly fragmented. Because instructions, target muscles, and equipment requirements are scattered across different websites and social media platforms, beginners often struggle to verify if two different names refer to the same movement or if an exercise suits their available gear.

Second, existing platforms lack an intuitive discovery mechanism. Without a structured taxonomy and a clean interface that pairs search filters with direct visual demonstrations, navigating large exercise libraries quickly becomes overwhelming.

Finally, there is a clear gap in content governance. A community-driven platform needs to leverage professional expertise without sacrificing data integrity. FiFB solves this by establishing separate User, Trainer, and Admin roles, complete with a dedicated moderation workflow to vet trainer contributions before they go live.

The practical significance of FiFB lies in providing a centralized environment for exercise information and controlled content contribution. Users can access structured exercise data, Trainers can contribute professional knowledge, and Administrators can manage account and content status. The REST-based architecture also provides a foundation that may be reused by future web or mobile clients.

1.3 Contribution and Position Regarding Novelty

At the GR1 stage, the project does not claim algorithmic or scientific novelty. Its contribution is primarily application-oriented, focusing on the integration of structured exercise data, role-based access control, modular architecture, and a normalized relational database.

The distinguishing direction of FiFB is the intended combination of beginner-oriented exercise discovery and community support. Existing products often specialize in workout logging, coach-business management, or curated training content. FiFB begins with a simpler knowledge and governance core that can later support trainer interaction, social features, and AI-assisted guidance without replacing the underlying user, exercise, and taxonomy modules. The contribution is consequently expressed through integration, scope discipline, and extensibility rather than through an unsupported claim of technical novelty.

1.4 Motivation and Development Orientation

The topic was selected because it aligns with current market trends, practical user needs, personal interest, and the potential usefulness of the resulting product. Fitness services are increasingly delivered through digital channels, while beginner users continue to require accessible explanations and trustworthy guidance. The project therefore represents a practical intersection between software engineering and an application domain with visible social relevance.

Personal experience with beginner fitness difficulties also contributed to the choice of topic, although this experience was limited and is not treated as formal empirical evidence. It nevertheless highlighted common questions such as how to identify an appropriate exercise, how to understand the relationship between an exercise and a muscle group, and how to obtain assistance when information from informal online sources is inconsistent. These observations motivated the construction of a product that prioritizes structured information and clear interaction flows.

The long-term aspiration is to develop FiFB into a fitness community in which users can be supported either directly by trainers or indirectly through the platform’s data and intelligent services. GR1 establishes the technical foundation; GR2 is expected to

investigate AI-assisted functions such as personalized exercise recommendations, conversational guidance, or other forms of context-aware support. These functions will only be introduced after their requirements, data quality, safety boundaries, and evaluation methods have been defined.

1.5 Objectives and Scope of GR1

The primary objective of GR1 is to design and implement a demonstrable web-platform foundation. The system must support account registration and login, identify the current actor, enforce role-based access for User, Trainer, and Admin, store exercise data in a relational model, provide administrative management of users and exercises, allow Trainer exercise submissions, and enable users to search, filter, and inspect exercise information. The objective also includes organizing the codebase so that later community and AI functions can be added without redesigning the complete application.

Table 1.1. Evaluated GR1 scope and explicit boundaries

Scope area	Included in GR1	Boundary
Authentication	Registration, login, token refresh, logout, basic account verification and recovery support	The report evaluates login and role access as the principal flows.
User role	Profile access, exercise search, taxonomy filters, pagination, and exercise details	Workout recommendation and AI coaching are not evaluated in GR1.
Trainer role	Basic trainer profile and submission of exercise content for review	Advanced trainer–client business workflows are outside the focused scope.
Admin role	User-status management, exercise creation and update, moderation, and taxonomy maintenance	The system does not manage a physical gym or individual gym machines.
Data layer	Users, trainer profiles, exercises, body parts, muscles, equipment taxonomy, and junction tables	Equipment represents exercise metadata rather than facility inventory.
Architecture	React client, Express API, PostgreSQL database hosted through Supabase, and modular code structure	Redis, Socket.IO, deployment automation, and other extensions are not required as GR1 evidence.

1.6 Research and Development Method

The project follows an iterative software-engineering method. The process begins with problem analysis and a survey of representative fitness products. The survey is used to identify interaction patterns, role models, data requirements, and usability characteristics that are relevant to beginner users. The resulting requirements are then translated into a client–server architecture, a relational schema, use cases, and role-specific interface flows.

Implementation proceeds from the data and backend foundation to the frontend interaction layer. Exercise data are acquired through the ExerciseDB V1 API Free Version, normalized, and persisted to a Supabase-hosted PostgreSQL database. Backend modules expose validated REST endpoints, while the React client consumes those endpoints through a centralized Axios layer. Verification is performed primarily through scenario-based functional demonstration. Automated testing libraries and linting tools are configured in the repository, but no complete benchmark dataset or formal performance measurement was supplied for this report; therefore, the evaluation remains transparent about its qualitative nature.

Table 1.2. Research and development stages

Stage	Main activity	GR1 output
Problem analysis	Identify beginner-user difficulties and actor responsibilities	Problem statement and scope boundary
Market survey	Examine fitness libraries, trackers, coaching systems, and interface approaches	Derived product and quality requirements
Data preparation	Acquire ExerciseDB data through API, normalize fields, and create taxonomy mappings	Persistent exercise dataset in Supabase PostgreSQL
System design	Define architecture, roles, database relationships, APIs, and interface structure	Design models and implementation plan
Implementation	Develop modular Express backend and React frontend	Working foundation demo
Evaluation	Execute principal user, trainer, and administrator scenarios	Completion assessment, limitations, and GR2 plan

1.7 Planned Roadmap from GR1 to GR2

Table 1.3. GR1–GR2 development roadmap

Phase	Primary emphasis	Expected outcome
GR1	Platform foundation, role model, relational database, authentication, user administration, exercise management, trainer submission, and exercise lookup	A stable and demonstrable web foundation with clearly defined scope
GR2	Refinement of community interaction, systematic testing, improved UI/UX, data-quality controls, and carefully scoped AI-assisted user support	An evaluated community-oriented product extension built on the GR1 architecture

This roadmap deliberately places artificial intelligence after the platform and data foundation. AI functions are meaningful only when user roles, data provenance, access control, and core interactions are sufficiently stable. The separation also allows GR1 to be evaluated against concrete software-engineering objectives rather than speculative future functions.

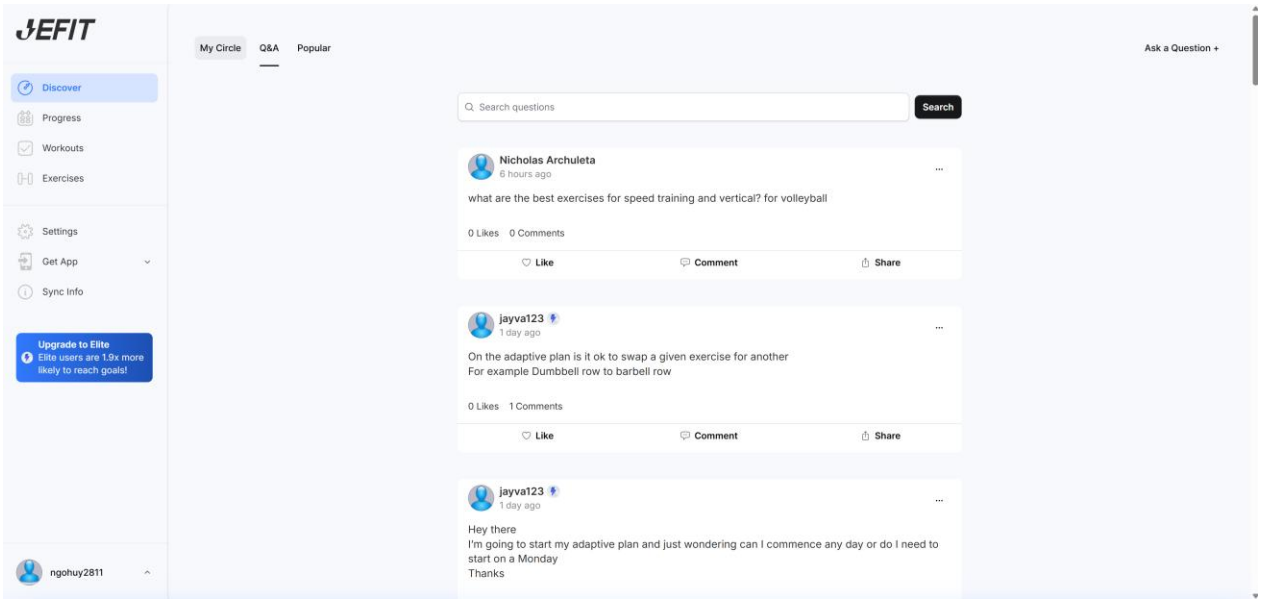
Chapter 2 – Survey and Related Products

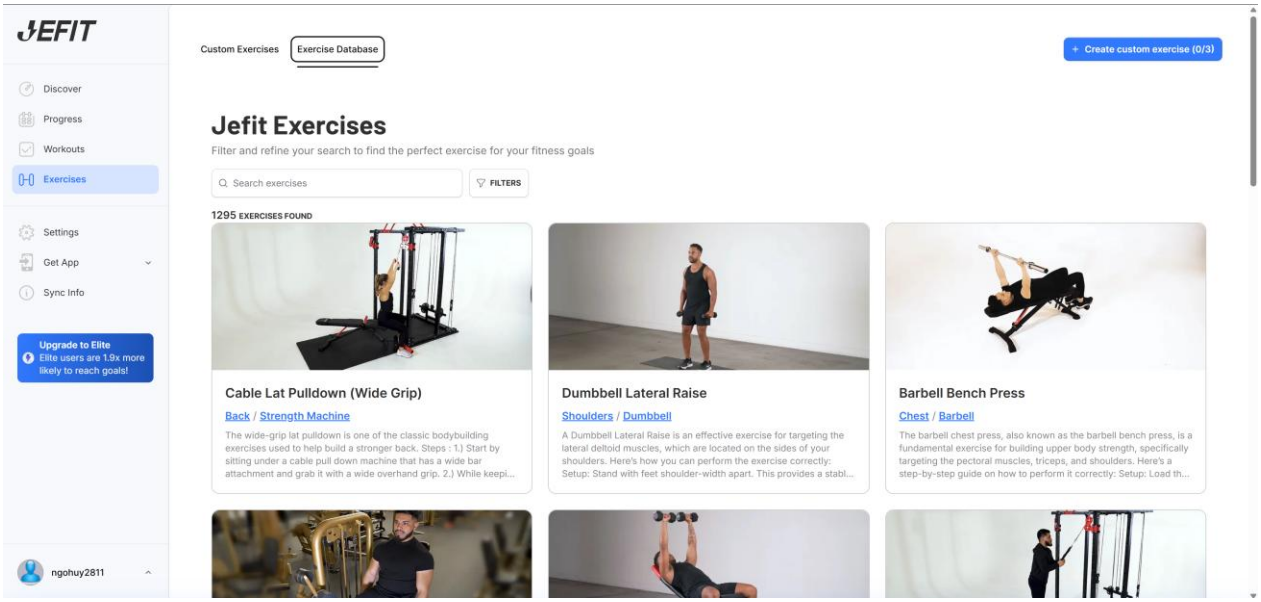
2.1 Survey Method and Evaluation Criteria

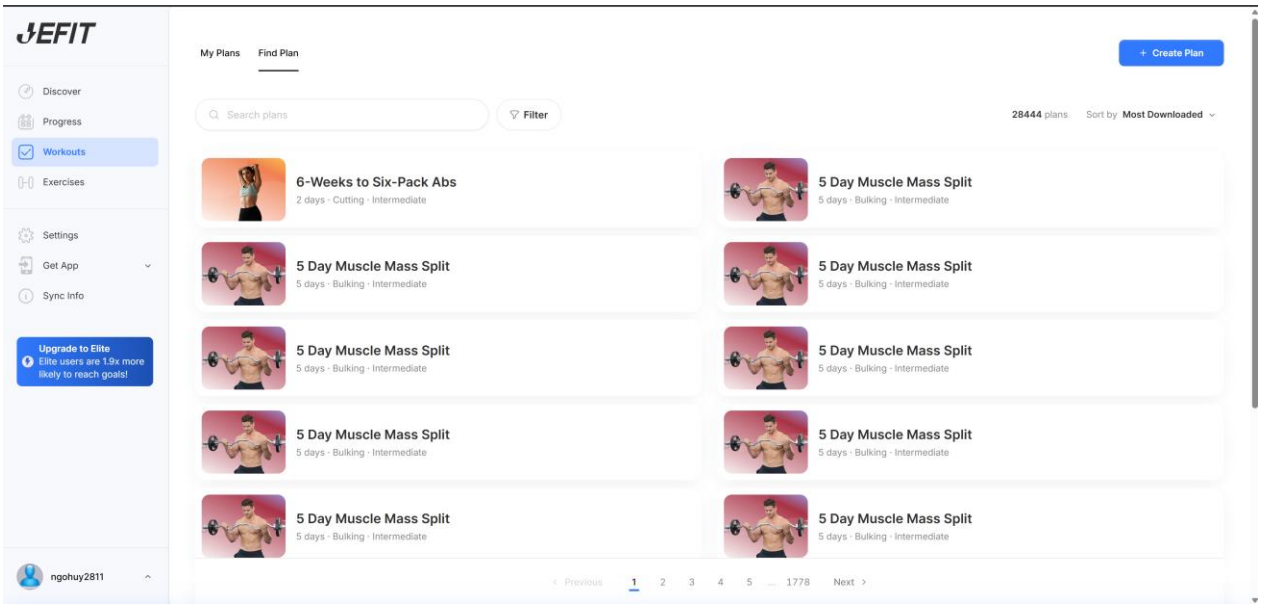
The market survey covers several products representing different parts of the digital fitness ecosystem. JEFIT is examined as a workout planner and exercise database, while Hevy represents a workout tracker with social features. ExerciseDB is considered separately as a data source rather than a direct competitor. The comparison focuses on both functional aspects, such as exercise search, workout tracking, trainer interaction, and community support, and frontend aspects, including navigation, visual presentation, information density, and ease of use for beginners [3]–[7].

2.2 JEFIT

JEFIT combines workout planning, exercise information, training logs, and progress tracking in a single platform [3]. Its large exercise library and structured workout data provide a useful reference for FiFB. However, the number of planning and tracking features may feel overwhelming to beginners who only need to understand how an exercise is performed, which muscles it targets, and what equipment it requires. FiFB should therefore make search, filtering, visual demonstrations, and concise exercise details available before introducing more advanced workout-planning functions.

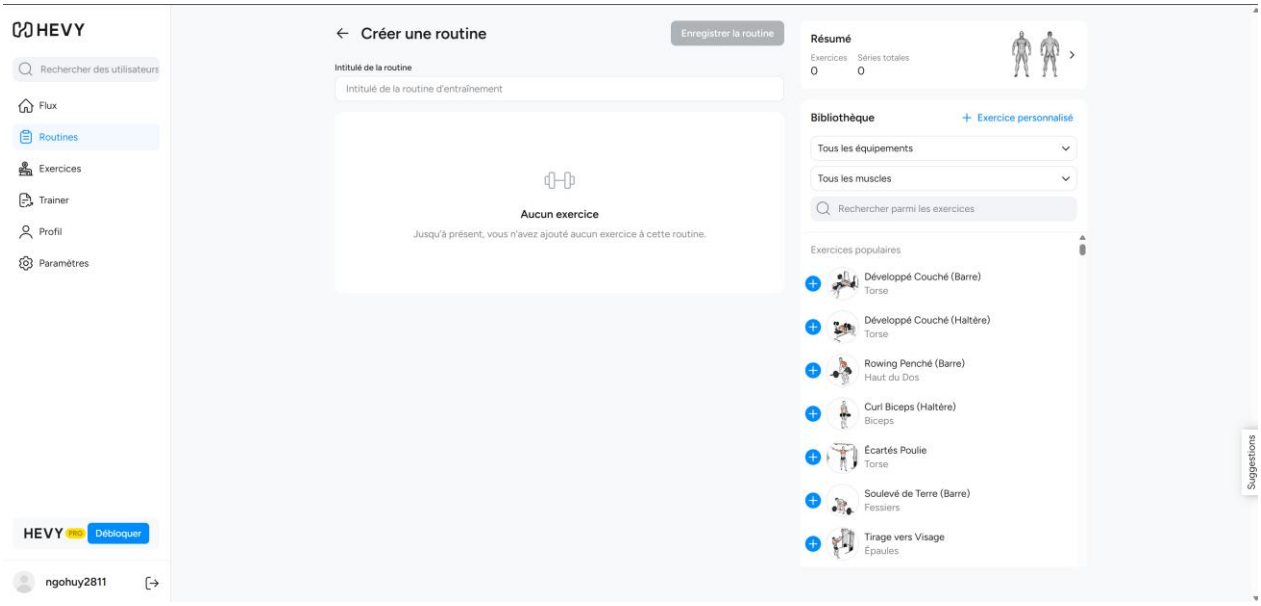


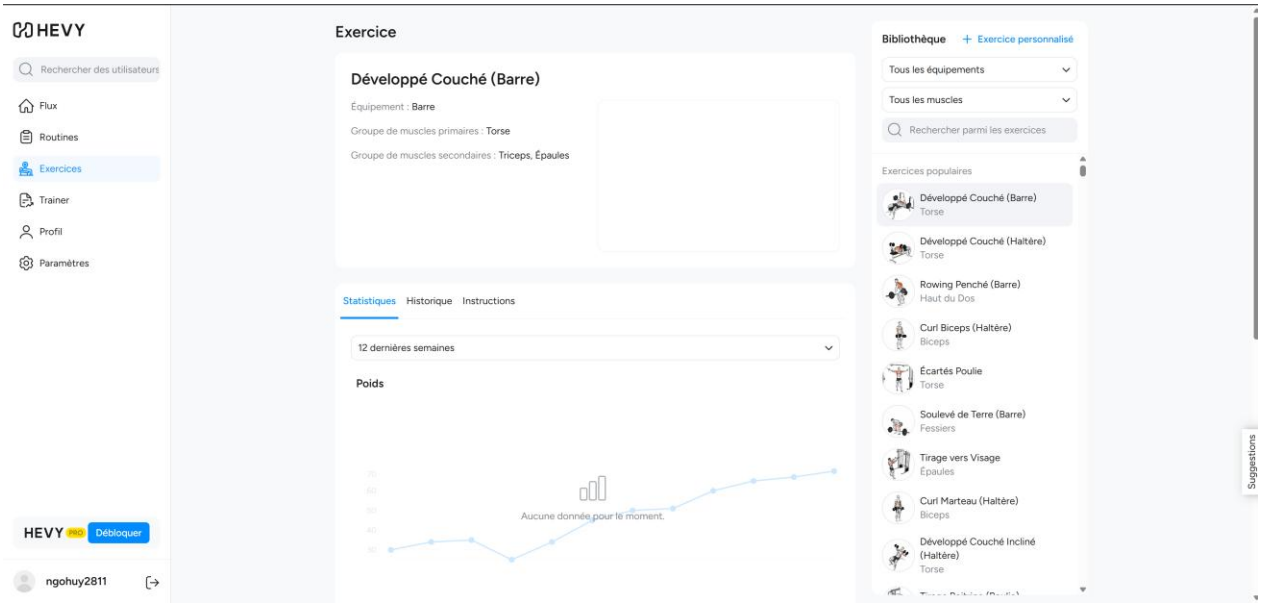




2.3 Hevy

Hevy focuses on simple workout logging, routine creation, progress statistics, and social interaction [4]. Its clean and mobile-friendly interface shows how community features can be added without making the main workflow difficult to understand. This direction is relevant to FiFB's long-term goal of becoming a fitness community. Nevertheless, because Hevy is primarily a workout tracker, FiFB should adopt its interface clarity and social orientation while keeping detailed training logs outside the main GR1 scope.





2.4 ExerciseDB as a Data Source

ExerciseDB V1 API Free Version provides a structured collection of more than 1,500 exercises with fields such as exercise name, animated GIF, body part, equipment, target muscle, secondary muscles, and instructions [7]. It is not an end-user community application and therefore does not provide FiFB's account management, moderation, role separation, or interface design. Its value lies in reducing the cost of manually creating the initial exercise dataset.

Using an external API does not eliminate the need for local data engineering. FiFB must define its own identifiers, normalize taxonomy values, preserve relationships, handle duplicate records, record source information, and remain operational independently of the

external response structure during ordinary user requests. For this reason, ExerciseDB data are imported into FiFB's PostgreSQL database rather than fetched directly every time a user opens the exercise library.

2.5 Functional Comparison

Table 2.1. Functional comparison of representative fitness products

Criterion	JEFIT	Hevy	FiFB
Primary purpose	Workout planning and tracking	Simple logging and social progress	Beginner discovery and fitness community foundation
Exercise library	Extensive library and routines	Library supporting workout logging	Structured exercise records with GIFs and taxonomy
Search and filtering	Rich but integrated with many features	Exercise selection within logging flows	Keyword, body-part, muscle, and equipment filters
Trainer interaction	Limited compared with coaching platforms	Community interaction rather than formal coaching	Basic Trainer role in GR1; broader support planned
Content governance	Platform-managed content	Platform-managed exercise catalog	Trainer submission with Admin moderation
Community orientation	Social and sharing features	Strong community emphasis	Long-term community objective
GR1 suitability	Reference for data depth	Reference for clarity and community	Focused integration of selected principles

2.6 Frontend and User-Experience Comparison

Table 2.2. Frontend and user-experience comparison

UX criterion	JEFIT	Hevy	FiFB
Information density	High due to planning, logs, and analytics	Moderate and focused on workout entry	Keep the GR1 navigation small and role-specific
Primary navigation	Feature-rich multi-function structure	Direct access to routines, logging, and community	Separate User, Trainer, and Admin layouts
Visual guidance	Exercise media and detailed data	Exercise visuals support logging	GIF and instructions must dominate the detail page
Beginner onboarding	Powerful but potentially complex	Relatively simple first-use flow	Allow public browsing and simple search before advanced actions
Social interaction	Available as a supporting feature	Integrated into the core experience	Introduce community features incrementally after the core library
Role visibility	Mostly consumer-oriented	Mostly consumer-oriented	Make permissions and available actions explicit in each layout

JEFIT provides a reference for data depth, Hevy for focused interaction and community motivation. FiFB combines only the elements that support its own problem statement. This selective approach prevents the demo from becoming a collection of disconnected features.

2.7 Derived Product Direction and Quality Attributes

The market survey leads to a product direction centered on discoverability, clarity, governed contribution, and extensibility. The exercise library must be accessible through keyword search and structured filters. The detail interface must connect the exercise name, visual demonstration, instructions, target muscles, body parts, and equipment in one coherent view. Trainer participation must be possible without granting unrestricted publication rights, and administrators must be able to manage account status and content lifecycle.

The most important software quality attributes are usability, security, maintainability, data integrity, reliability, and extensibility. Usability requires role-specific navigation, visible loading and error states, and a low-complexity discovery flow. Security requires password hashing, authenticated requests, and backend authorization rather than frontend-only route hiding. Maintainability requires modular boundaries and reusable components. Data integrity requires normalized relationships and database constraints. Reliability requires predictable error responses and transaction control. Extensibility requires that future community and AI functions use the same stable identity, role, and exercise-data foundation.

For GR1, the product is therefore considered successful when the foundation can demonstrate the principal role flows and exercise discovery process. Features such as complete workout tracking, social feeds, real-time chat, and AI guidance remain valuable to the product vision but are not prerequisites for the current academic milestone.

Chapter 3 – Theoretical Foundations and Technology Stack

3.1 End-to-End Technical Workflow

The technical workflow begins after the problem and scope have been identified. The first engineering task is to determine the information required by the application. For FiFB, an exercise record must contain a stable identifier, name, visual demonstration, ordered instructions, source, lifecycle status, and relationships to body parts, equipment, target muscles, and secondary muscles. This information model drives both the data-acquisition process and the relational database design.

The second task is to obtain a sufficiently broad initial dataset. Instead of manually entering hundreds of exercise records, the project uses ExerciseDB V1 API Free Version. The source data are requested page by page, normalized into the FiFB schema, and stored in a Supabase-hosted PostgreSQL database. Once the local dataset exists, the user-facing application no longer depends on a live ExerciseDB request for each search. The backend queries FiFB's own tables and applies authorization, status rules, filters, and pagination.

The third task is system design. The frontend and backend are separated into independent applications connected through HTTP. The backend is decomposed into modules and layers, while the database schema is normalized around stable domain entities. The frontend is decomposed into pages, layouts, routes, reusable components, API clients, and authentication state. This design is then implemented incrementally, beginning with authentication and common infrastructure before role-specific functions are added.

The final task is verification. Core flows are examined from the user interface to the database: a login request is validated and converted into an authenticated session; an administrator action is checked against the current role; an exercise search is translated into parameterized SQL; and the response is rendered as loading, success, empty, or error states. This end-to-end perspective is important because a function is not complete merely because one interface page or one API route exists in isolation.

3.2 Client–Server Architecture and RESTful Communication

FiFB follows a client–server architecture. The React application is responsible for presentation, navigation, local interaction state, and communication with the backend. The Express application exposes business capabilities through HTTP endpoints, validates requests, enforces permissions, and accesses persistent data. PostgreSQL, hosted through Supabase in the current deployment, stores the authoritative application state. The

separation reduces direct coupling between the user interface and database structure and allows either side to evolve behind a stable API contract.

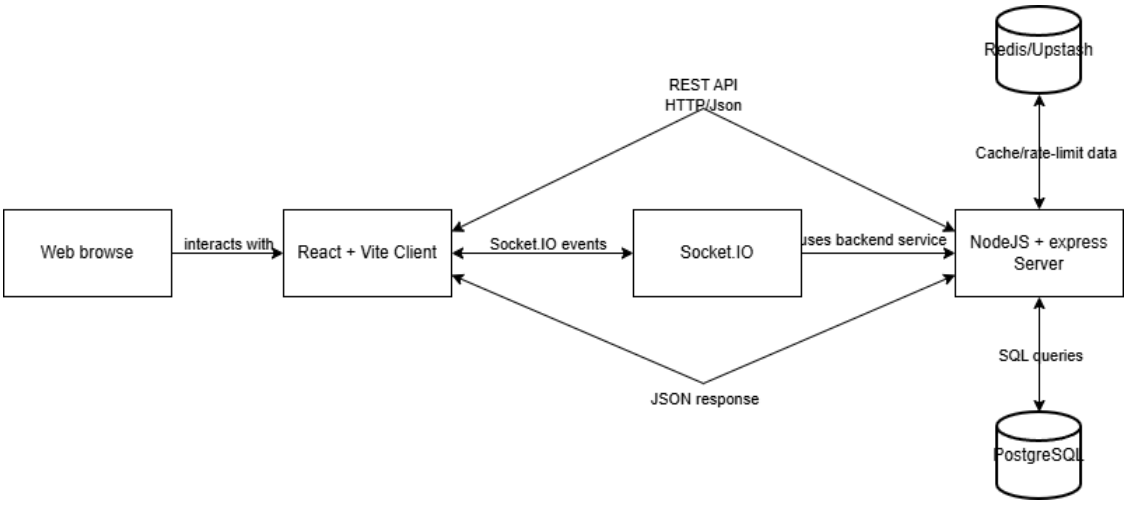


Figure 3.1. Designed client–server architecture of the FiFB repository

Figure 3.1 presents the overall system architecture, while the FiFB implementation mainly follows the React and Vite client → Node.js and Express server → PostgreSQL flow. Redis, Upstash, and Socket.IO are reserved for future features such as caching, notifications, and real-time communication.

The backend exposes RESTful APIs for authentication, users, trainers, exercises, and exercise taxonomy. Data are exchanged in JSON format, and list endpoints use pagination to improve performance and avoid loading the entire exercise dataset at once [17].

3.3 Frontend Technology Stack

React is used to build the main interfaces for authentication, exercise discovery, trainer activities, and administration [8]. React Router manages navigation, Axios handles API requests, and React Hook Form supports form processing [10], [11]. Tailwind CSS provides responsive styling, while Lucide React and Sonner support icons and notifications [12], [24].

This stack is suitable for the GR1 prototype because it supports rapid development while maintaining clear separation between pages, components, state management, and API communication.

3.4 Authentication, JWT, and Password Protection

FiFB uses email and password authentication. Passwords are hashed with bcryptjs before being stored, while JSON Web Tokens are used to identify authenticated accounts

and their roles [18], [19]. Protected requests include the access token in the Authorization header, and backend middleware verifies the token before allowing access to restricted functions.

The Axios client automatically attaches access tokens and attempts to refresh expired sessions [11]. When token renewal fails, the application clears the current session and redirects the user to the login page. Although frontend route guards improve navigation, actual security is enforced by the backend API.

3.5 Role-Based Access Control

FiFB defines three main roles: User, Trainer, and Admin. Users access exercise information and personal functions, Trainers can manage their profiles and contribute exercise content, while Admins are responsible for user accounts, exercise records, and content moderation.

Access control is applied at both backend and frontend levels. Backend middleware and service rules prevent unauthorized operations, while the frontend displays only the routes and actions available to the current role. Database constraints provide an additional level of protection for valid roles, statuses, and relationships.

Table 3.1. Core role model

Actor	Core GR1 permissions	Explicit restriction
Visitor	Browse public exercises, inspect details, register, and log in	Cannot access profile or management operations
User	Manage own profile and use exercise discovery functions	Cannot manage other accounts or publish administrative content
Trainer	Use exercise discovery, maintain a trainer profile, and submit exercise content	Cannot directly approve submissions or administer users
Admin	Manage account status, exercise lifecycle, moderation, and taxonomy data	Administrative power remains subject to validation and auditability

3.6 Relational Database Design and Supabase Hosting

PostgreSQL is used because FiFB contains several related entities, including users, trainer profiles, exercises, body parts, muscles, and equipment categories [15]. Many-to-many relationships are represented through junction tables, while foreign keys and constraints maintain consistency between records. The backend uses the pg driver and explicit SQL queries, which makes the database interactions clear and suitable for the objectives of GR1.

The database is hosted on Supabase, but the Express server remains responsible for business logic and authorization [16]. Exercise taxonomy is normalized into separate

tables to reduce duplication and support filtering. UUID identifiers, status fields, indexes, parameterized queries, and transactions are used to improve consistency and maintainability.

3.7 ExerciseDB V1 API Data Acquisition and Processing

The initial exercise dataset was collected from the ExerciseDB V1 API Free Version and successfully stored in the Supabase PostgreSQL database. The process uses structured API responses rather than HTML scraping, although it may still be referred to informally as data crawling.

The import script retrieves records in pages, validates essential fields, and updates or inserts exercises using their external identifiers. Body parts, equipment, target muscles, and secondary muscles are normalized and linked through the appropriate junction tables. Transactions ensure that an exercise and its related taxonomy mappings are updated consistently.

The script also records successful, skipped, updated, and failed items. Because the exact imported record count was not preserved, the report confirms the successful import without presenting an unsupported numerical result.

3.8 Backend Technology Stack

The backend is implemented with Node.js and Express.js [13], [14], [24]. Its modular structure separates routes, controllers, services, repositories, and validation schemas, making the code easier to maintain and extend.

PostgreSQL access is handled through pg, while Zod validates incoming data [20]. JWT and bcryptjs support authentication, Helmet and CORS improve API security, and logging, compression, and rate-limiting libraries support system operation. Redis, Socket.IO, and SendGrid belong to the broader system design and are treated as future extension technologies rather than core GR1 requirements.

3.9 Validation, Testing, and Code-Quality Support

The repository includes ESLint and Prettier for code consistency, Jest for unit testing, and Supertest for API integration testing [22]–[24]. These tools provide a basic quality-assurance foundation for both frontend and backend development.

3.10 Technology Summary and Scope Boundary

Table 3.2. Technology stack summary

Layer	Technology	Repository version or role
Frontend	React	18.3.1; component-based UI
Frontend build	Vite	6.0.5; development server and production build
Routing	React Router DOM	6.28.1; public, protected, and role routes
HTTP client	Axios	1.7.9; API calls and token interception
Forms and UI	React Hook Form, Tailwind CSS, Lucide React, Sonner	Form state, responsive styling, icons, and notifications
Backend runtime	Node.js	Version 20 or later
Backend framework	Express.js	4.21.2; routes and middleware
Database	PostgreSQL hosted through Supabase	Relational persistence accessed through pg
Security	JWT, bcryptjs, Helmet, CORS, rate limiting	Authentication and defensive controls
Validation	Zod	Request schema validation
Quality support	ESLint, Prettier, Jest, Supertest	Static checks and automated-test capability
Extensions	Redis/Upstash, Socket.IO, SendGrid, Docker, CI/CD	Broader repository support; not core GR1 evidence

Chapter 4 – Design and Development

4.1 Product Overview and Actors

FiFB is a web-based fitness platform intended to evolve into a community that supports beginner gym users. In the GR1 implementation, the product centers on a structured exercise library and a controlled contribution model. Public visitors can inspect exercise information and create accounts. Authenticated users receive profile and discovery functions. Trainers obtain an additional profile and exercise-submission workflow. Administrators govern accounts, exercise content, and taxonomy data.

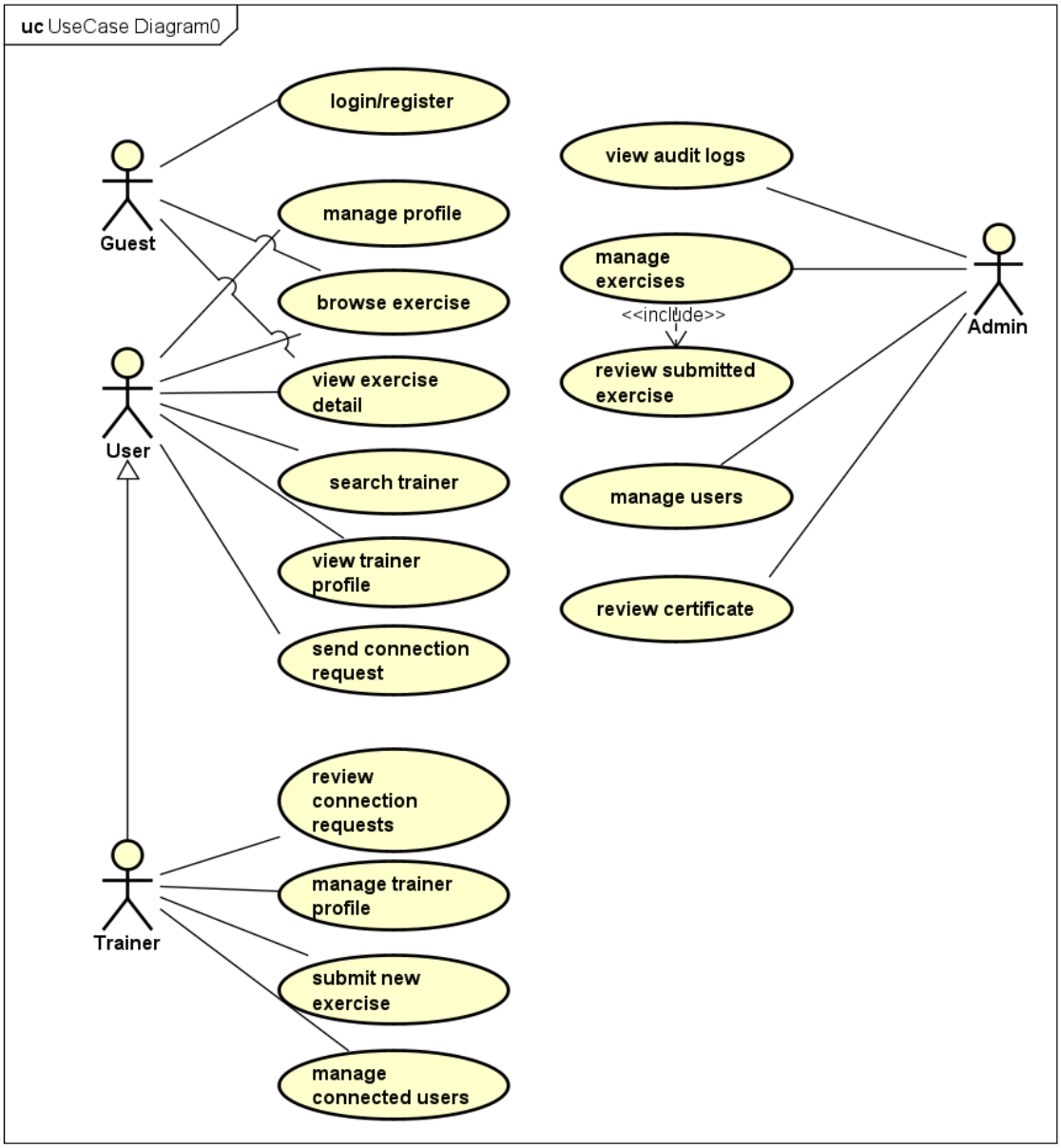


Figure 4.1. Use-case diagram created for the broader FiFB product design

The focused GR1 implementation and evaluation use only the subset associated with authentication, profiles, exercise discovery, trainer submission, user management, exercise moderation, and administrative governance.

4.2 Functional Requirements

Functional requirements specify the observable behavior expected from the system. They are grouped by actor because the same resource may be visible or editable under different rules. The descriptions below emphasize complete business flows rather than isolated interface buttons.

4.2.1 Visitor Requirements

An unauthenticated visitor must be able to access public authentication pages and browse active exercise content. The visitor can search and filter the exercise library and open a detail page containing the visual demonstration, instructions, body-part associations, muscles, and equipment. Registration must allow the creation of a User or Trainer account, after which the account can participate in the appropriate authenticated flows. A visitor must not be able to access another user's profile or any management endpoint.

Table 4.1. Visitor functional requirements

ID	Requirement	Acceptance description
G-01	Browse active exercises	The public library returns only content eligible for ordinary viewing.
G-02	Search and filter	Keyword and taxonomy parameters can be combined with pagination.
G-03	View exercise details	The detail page connects GIF, instructions, muscles, body parts, and equipment.
G-04	Register and log in	A visitor can create an account and establish an authenticated session.

4.2.2 User Requirements

An authenticated User must be able to view and update permitted profile fields and use the exercise library without administrative complexity. Search results must respond to the current keyword, filter combination, and page. Selecting an exercise must open the complete information view. The user interface must expose only user-relevant navigation and must not display controls that imply authority to manage other accounts or publish content.

Table 4.2. User functional requirements

ID	Requirement	Acceptance description
U-01	Manage own profile	The account can retrieve and update allowed personal fields.
U-02	Search exercises	The system matches active exercise names and supported search fields.
U-03	Filter exercises	Body part, target muscle, secondary muscle, and equipment filters are supported.
U-04	View exercise details	The page displays structured instructions and taxonomy associations.
U-05	Use role-appropriate interface	User routes and actions exclude Trainer- and Admin-only operations.

4.2.3 Trainer Requirements

A Trainer retains access to the shared exercise-discovery functions and gains a trainer-profile workflow. The profile can store information such as biography, specialization, and years of experience. A Trainer can submit a proposed exercise with the information needed for review. The submission must enter a pending state rather than becoming immediately visible as approved content. This requirement balances community contribution with platform governance.

Table 4.3. Trainer functional requirements

ID	Requirement	Acceptance description
T-01	Maintain trainer profile	The Trainer can create or update professional profile information.
T-02	Use exercise library	The Trainer can search, filter, and inspect the shared content.
T-03	Submit exercise content	A new proposal is stored for administrative review.
T-04	Use Trainer interface	Trainer-only pages are protected from ordinary User accounts.

4.2.4 Admin Requirements

An Admin must be able to retrieve a filtered and paginated user list, inspect an account, and change its lifecycle status. Administrative user management is status-based so that an account can be locked or disabled without deleting its history. The Admin must also be able to create and update exercises, review Trainer submissions, approve or reject content, and deactivate records that should no longer be available to ordinary users.

Taxonomy management supports the consistency of exercise metadata. Body parts, muscles, and equipment categories can be listed and maintained so that search filters use canonical values. Equipment management in this context is exclusively taxonomy management.

Table 4.4. Admin functional requirements

ID	Requirement	Acceptance description
A-01	Manage user status	The Admin can list, filter, inspect, lock, unlock, or disable accounts.
A-02	Create and update exercises	Administrative forms can maintain core content and taxonomy mappings.
A-03	Moderate submissions	Pending Trainer content can be approved or rejected.
A-04	Deactivate exercise content	Inactive content is excluded from ordinary public results.
A-05	Maintain exercise taxonomy	Canonical body-part, muscle, and equipment values support consistent filtering.
A-06	Use Admin interface	Administrative pages and API operations require the Admin role.

4.3 Non-Functional Requirements

Non-functional requirements define the quality constraints under which the functional behavior should operate. Because the present system is a GR1 demonstration, quantitative service-level objectives have not yet been measured. The requirements are therefore written as design and verification targets.

Table 4.5. Non-functional requirements

Quality attribute	Requirement for the GR1 foundation
Usability	Role-specific navigation, consistent forms, visible loading/error/empty states, and a direct exercise-discovery flow should minimize unnecessary steps.
Security	Passwords must be hashed; protected API operations must validate JWT and role; request data must be validated; browser origins must be controlled.
Data integrity	Primary keys, foreign keys, uniqueness rules, valid statuses, transactions, and normalized junction tables must prevent inconsistent records.
Maintainability	Backend responsibilities must be separated into routes, controllers, services, repositories, and validation; frontend responsibilities must be separated into API, pages, layouts, routes, contexts, hooks, and components.
Reliability	Unexpected routes and operational errors must produce controlled responses; one failed component or request should not make the complete application unusable.
Performance	List operations must use pagination and database queries should use indexes and bounded filtering; formal latency benchmarks remain future work.
Extensibility	Community, real-time, and AI functions should be addable without replacing the core identity, exercise, taxonomy, and authorization modules.

Quality attribute	Requirement for the GR1 foundation
Portability	Environment variables and standard web technologies should allow local, containerized, or managed-database deployment.

4.4 System Architecture and Project Structure

The core architecture contains three runtime layers. The React and Vite client renders the user interface and communicates through JSON over HTTP. The Express server receives requests, applies middleware and business rules, and returns standardized responses. The Supabase-hosted PostgreSQL database stores accounts, profiles, exercises, taxonomy, and relationships. The backend is the only application layer that should execute business-sensitive database operations for the core web client.

The backend adopts a layered modular pattern. Routes define endpoint paths and middleware composition. Controllers translate HTTP input into service calls and format output. Services enforce business rules, lifecycle transitions, and authorization decisions. Repositories contain parameterized SQL and database mapping logic. Validation files define request schemas. This separation keeps HTTP concerns, domain decisions, and persistence logic from becoming a single monolithic module.

```
server/src/
├── config/                 Environment and application configuration
├── db/                     PostgreSQL pool, transactions, and database support
├── middlewares/            Authentication, roles, validation, errors, and
security
├── modules/
│   ├── auth/               Registration, login, tokens, password workflows
│   ├── users/              Own profile and Admin user management
│   ├── trainers/           Trainer profile and core Trainer functions
│   ├── exercises/          Exercise discovery and content lifecycle
│   ├── exerciseTaxonomy/   Body parts, muscles, and equipment metadata
│   ├── admin/              Administrative review operations
│   └── ...                 Community and extension modules
├── redis/                  Optional caching and distributed support
├── routes/                 Root route registration
├── socket/                 Optional real-time layer
├── utils/                  Shared helpers
├── app.js                  Express application composition
└── server.js               Process startup and server lifecycle
```

The frontend follows a feature-oriented presentation structure. API modules isolate endpoint details, while route components determine authentication and role access. Layouts provide different navigation frames for User, Trainer, and Admin. Page folders organize actor-specific screens, and reusable components maintain consistent behavior for tables, forms, feedback, and pagination.

```
client/src/
├── api/                    Axios client and domain API functions
├── components/             Reusable UI components
├── contexts/               Authenticated account and shared state
├── hooks/                  Reusable asynchronous and UI behavior
├── layouts/                Public, User, Trainer, and Admin layouts
├── pages/
│   ├── auth/               Login, registration, verification, recovery
│   ├── user/               Exercise library, details, and profile
│   ├── trainer/            Trainer profile and exercise submission
│   └── admin/              User, exercise, taxonomy, and support pages
├── routes/                 AppRoutes, ProtectedRoute, and RoleRoute
├── styles/                 Global styling support
├── utils/                  Client-side utility functions
├── App.jsx                 Application root
└── main.jsx                React bootstrap
```

4.5 Database Design

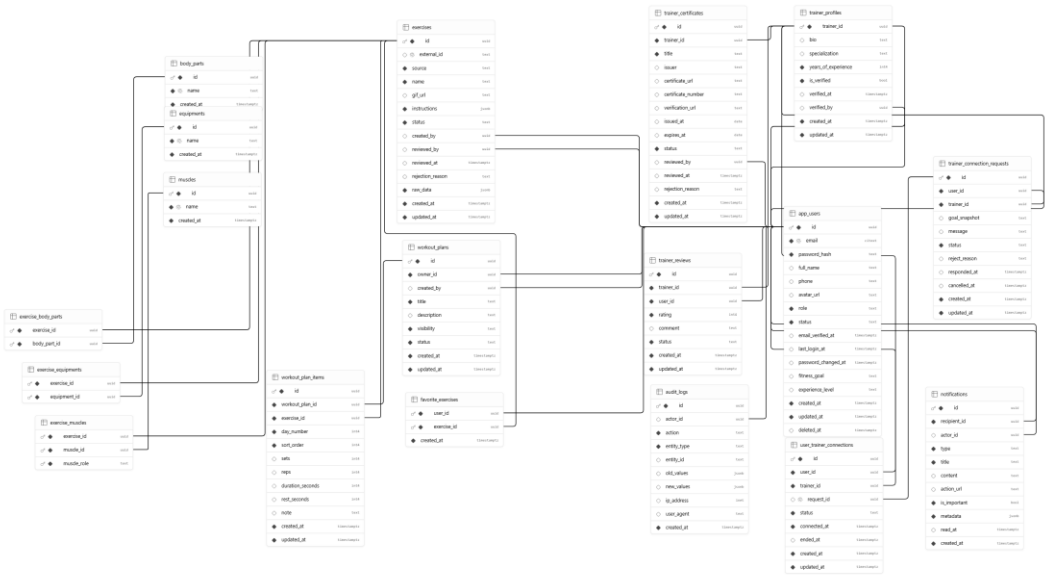


Figure 4.2. Entity–relationship diagram of the broader FiFB repository

The app_users table is the identity root. It stores email, password hash, role, account status, and basic profile data. trainer_profiles forms a one-to-one role extension so Trainer-specific attributes do not make every User record sparse. The exercises table stores the imported or administratively created exercise content, including source, external identifier, name, GIF URL, ordered instructions, status, and raw source data.

Body parts, equipment, and muscles are represented as independent taxonomy tables. exercise_body_parts and exercise_equipments model many-to-many relationships. exercise_muscles includes the semantic role target or secondary, which allows one muscle table to represent both categories without duplicating names. audit_logs provide traceability for sensitive administrative changes. Additional tables for favorites, workout

planning, workout sessions, connections, certificates, reviews, chat, and notifications belong to the broader product design and are not necessary to prove the core GR1 flows.

Table 4.6. Core GR1 database entities

Core table	Responsibility	Principal relationships
app_users	Authentication identity, role, status, and basic profile	Referenced by trainer profile, submissions, and administrative history
trainer_profiles	Trainer-specific biography, specialization, and experience	One-to-one extension of app_users
exercises	Exercise content, media, instructions, source, and lifecycle	Connected to taxonomy through junction tables
body_parts	Canonical body-part classification	Many-to-many with exercises
equipments	Canonical equipment category required by an exercise	Many-to-many with exercises; not physical inventory
muscles	Canonical muscle names	Many-to-many with exercises through target/secondary roles
exercise_body_parts	Exercise-to-body-part mapping	Composite relationship table
exercise_equipments	Exercise-to-equipment mapping	Composite relationship table
exercise_muscles	Exercise-to-muscle mapping with semantic role	Composite relationship table
audit_logs	Traceability for sensitive administrative changes	References actor and affected operation where applicable

4.6 Backend Design and Implementation

The FiFB backend is organized into domain modules for authentication, users, trainers, exercises, exercise taxonomy, and administrative operations. Each module follows a layered structure consisting of routes, controllers, services, repositories, and validation schemas. Instead of describing every endpoint separately, this section focuses on the main implementation flows and the technical decisions required to connect authentication, external exercise data, content moderation, relational search, and database consistency.

4.6.1 API Resource Groups

The backend exposes resource groups for authentication, users, trainers, exercises, exercise taxonomy, and administrative operations. Authentication routes support registration, verification, login, current-user retrieval, token refresh, logout, password change, and password recovery. The GR1 evaluation gives greatest weight to login, current-user retrieval, and role enforcement, while the additional endpoints complete a practical account lifecycle.

Table 4.7. Core backend resource groups

Resource group	Representative operations	Role significance
Auth	Register, verify, login, refresh, logout, current account, password workflows	Public entry and authenticated session lifecycle
Users	Read/update own profile; list, inspect, and update account status	User self-service and Admin governance
Trainers	Read/update Trainer profile and submit exercise content	Trainer-specific contribution flow
Exercises	List, filter, read detail, create, update, review, reject, deactivate	Shared discovery and administrative lifecycle
Exercise taxonomy	List and maintain body parts, equipment, and muscles	Canonical metadata for filters and exercise relationships
Admin support	Moderation, traceability, and selected management operations	Restricted platform governance
Health	Return service availability status	Operational verification

4.6.2 Authentication and User Access Management

The authentication flow begins when the frontend sends the user's email and password to POST /api/auth/login. After request validation, the authentication service retrieves the account by email and compares the submitted password with the stored bcrypt hash. Login is accepted only when the password is correct, the account is active, and the email has been verified. The backend then creates an access token and a refresh session for the authenticated account.

Protected requests include the access token in the authorization header. The authentication middleware verifies the token and reloads the corresponding account from the database. Additional middleware checks whether the account remains active, whether its email has been verified, and whether its role is allowed to perform the requested operation. Frontend route guards improve navigation by hiding unavailable pages, but the backend remains responsible for the final access decision.

User administration also follows a status-based approach. Rather than deleting an account immediately, an Admin can change its status so that access is restricted while the account and its related records remain available for administrative review. This implementation connects account management directly with the authorization process because a previously issued token does not allow a locked or disabled account to bypass the current database status. Authentication routes apply validation and account checks, while protected routes use authenticate, requireActiveUser, requireVerifiedEmail, and requireRoles.

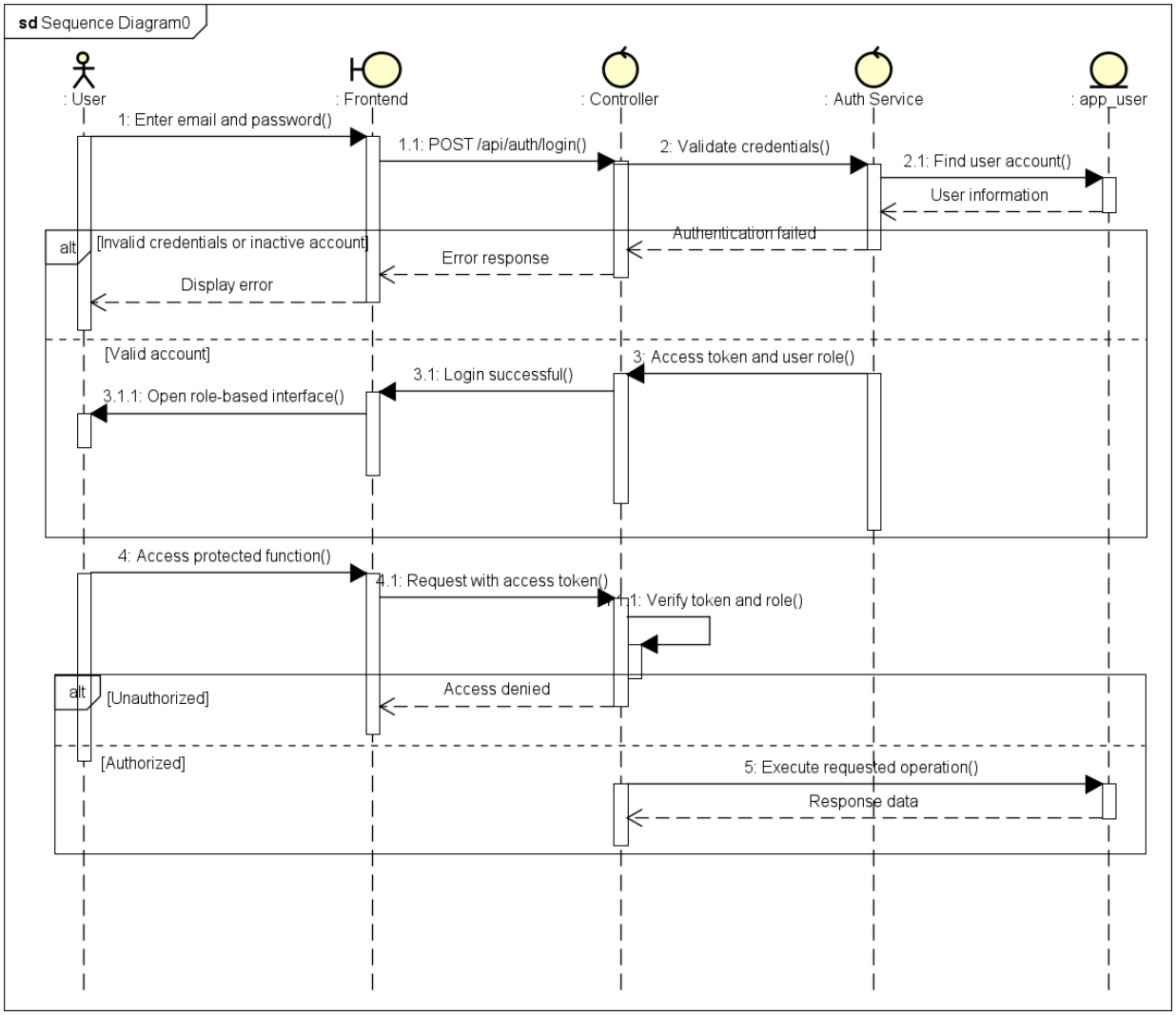


Figure 4.3. Sequence diagram for authentication and role-based access management.

4.6.3 Exercise Data Acquisition and Normalization

The initial exercise dataset was imported from the ExerciseDB V1 API Free Version through a dedicated script. Because the API contains a large number of records, the script retrieves data page by page and supports both page-based and cursor-based responses. Each record is checked for an external identifier and exercise name before being processed.

The external exercise identifier is used to prevent duplicate records. When an identifier does not exist in FiFB, a new exercise is inserted. When the same identifier is found during a later import, the existing record is updated instead of being inserted again. This allows the import script to be executed repeatedly when the external data change without creating multiple copies of the same exercise.

ExerciseDB taxonomy values are also converted into FiFB's relational structure. Body parts, equipment, target muscles, and secondary muscles are normalized before being stored in their respective tables. The script then creates the corresponding exercise-taxonomy relationships. Each exercise is processed inside a database transaction so that the exercise record and its mappings are either committed together or rolled back together when an error occurs. The script records created, updated, skipped, and failed operations to provide a basic summary of the import result.

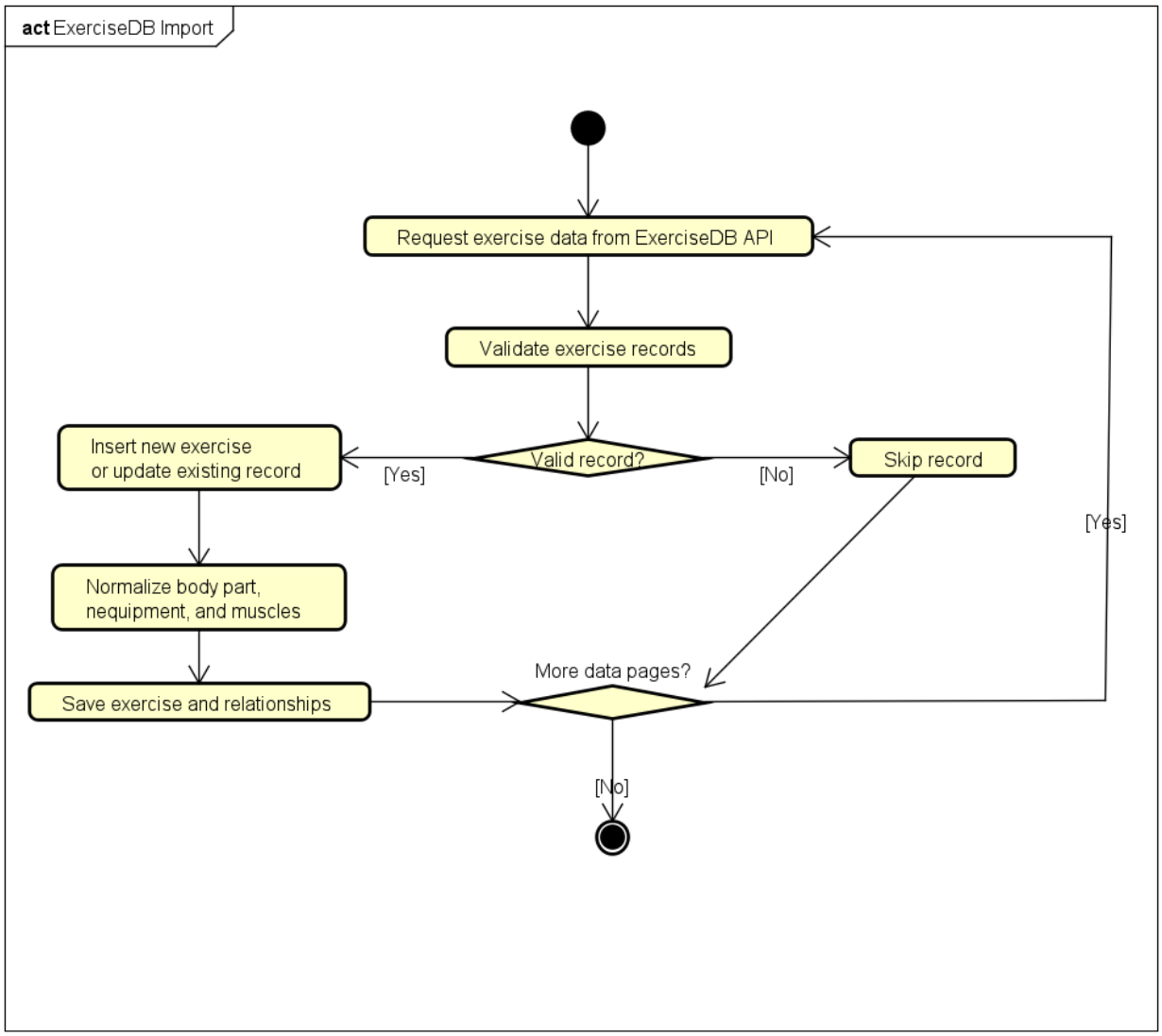


Figure 4.4: ExerciseDB Import Activity Diagram

4.6.4 Exercise Content Lifecycle and Moderation

FiFB separates exercise contribution from public visibility. Both Admin and Trainer accounts can send exercise data through the exercise-creation endpoint, but the resulting status depends on the source of the request. An exercise submitted by a Trainer is stored

with a pending status, while an exercise created directly through the administrative flow may become active immediately.

Pending content is reviewed through an Admin-protected endpoint. The Admin can approve or reject the submission. Approval changes the exercise status to active, allowing it to appear in the public library, whereas rejection preserves the record together with its review result. An active exercise can later be changed to inactive instead of being physically deleted. This approach preserves content history and prevents dependent taxonomy relationships from being removed unnecessarily.

Exercise creation and update may affect several tables because an exercise can be associated with multiple body parts, equipment categories, and muscles. These operations are therefore performed within a transaction. Relevant administrative changes are also recorded through the audit mechanism, providing traceability for sensitive content-management operations [24]. In the current source code, POST /api/exercises is available to Admin and Trainer roles; Trainer-originated records are assigned pending, while Admin review can change them to active or rejected.

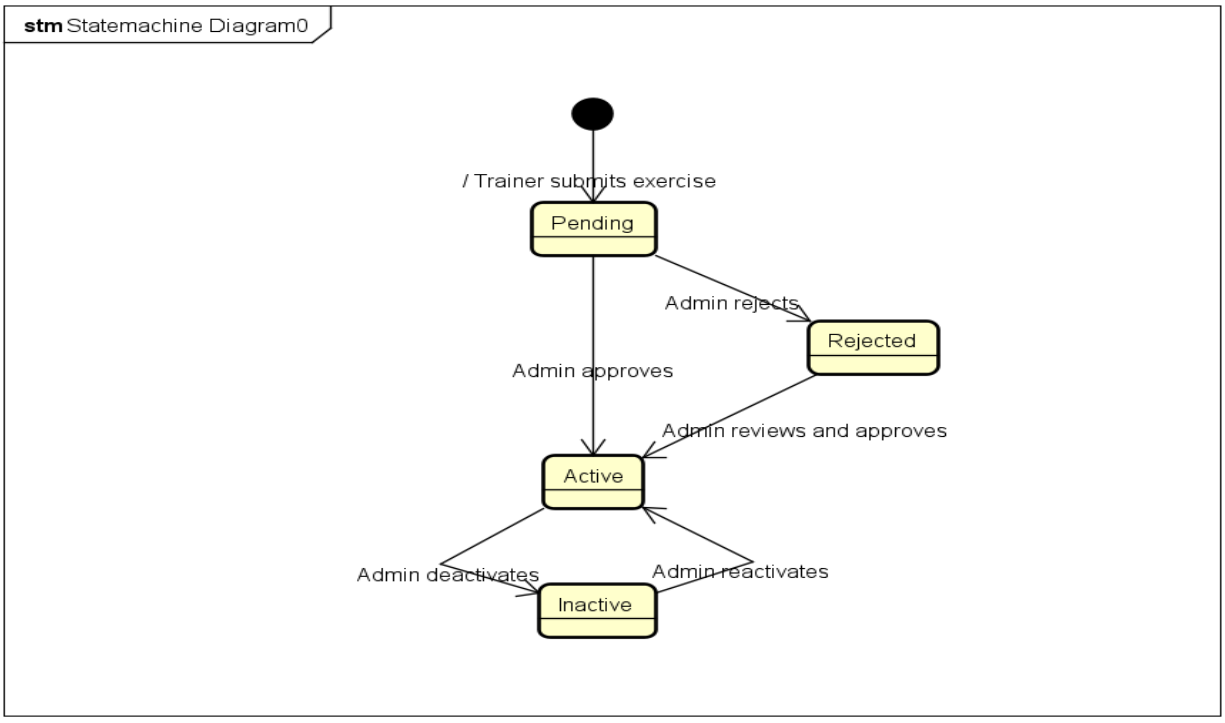


Figure 4.5: Exercise lifecycle state diagram.

4.6.5 Exercise Search and Relational Data Retrieval

The public exercise library is accessed through GET /api/exercises. The frontend sends the current keyword, taxonomy filters, page, and page size as query parameters.

Before the request reaches the repository, the parameters are validated to prevent unsupported filter values or invalid pagination input.

The repository returns only active exercises for ordinary public requests. Keyword search is applied to the exercise name, while body-part, equipment, target-muscle, and secondary-muscle filters are resolved through the corresponding junction tables. Target and secondary muscles are distinguished by the role stored in the exercise-muscle relationship.

The backend first determines the total number of matching records and then applies LIMIT and OFFSET to retrieve the requested page. Related taxonomy records are aggregated into the returned exercise objects so that the frontend receives the exercise and its body parts, equipment, target muscles, and secondary muscles in one response. This allows the exercise-library and detail pages to display structured information without issuing a separate request for every taxonomy relationship.

4.6.6 Validation, Transactions, and Error Handling

Request validation is applied to body data, route parameters, and query parameters before the corresponding controller is executed. Authentication requests, account-status updates, exercise filters, exercise payloads, taxonomy values, and UUID parameters therefore use predefined validation schemas rather than relying on manual checks inside every controller.

Repositories execute parameterized SQL so that user-provided values are transmitted separately from the SQL statement. Operations that modify an exercise together with its taxonomy relationships use database transactions to prevent partial updates. The same principle is applied during data import and exercise review, where several related operations must succeed as one unit.

Operational failures are passed to centralized error-handling middleware. Authentication, authorization, validation, not-found, conflict, and database errors are converted into a consistent response structure before being returned to the frontend. This allows the user interface to display predictable validation messages, permission errors, retry states, and general server failures without implementing a different error format for every backend module [24]. Validation is mounted directly in the route definitions, and transaction-based service operations are used for exercise creation, update, review, and import.

4.7 Frontend Design and Implementation

4.7.1 Routing and Layout Separation

AppRoutes defines public, authenticated, and role-specific navigation. ProtectedRoute verifies that an authenticated account exists before rendering protected content, while RoleRoute checks whether the current role is included in the allowed set. Authenticated users are presented with layouts and navigation appropriate to User, Trainer, or Admin. The route structure therefore mirrors the backend role model and reduces the likelihood that an actor will be confused by unavailable actions.

4.7.2 API Client and Authentication State

The API layer separates page components from raw endpoint details. Each domain module exports functions for the requests used by its pages. A shared Axios instance defines the API base URL, credentials behavior, access-token injection, refresh handling, and normalized client errors. AuthContext stores the current account, login and logout operations, current-user reload behavior, and role-check helpers. Pages consume this context instead of independently parsing tokens or duplicating session rules.

4.7.3 Search, Filters, and Exercise Details

The exercise-library page maintains the current keyword, selected taxonomy filters, page, loading state, result set, and total-page information. A change in search conditions produces a bounded API request. The response is rendered as exercise cards or rows with clear empty and error states. The detail page presents the GIF, instructions, body parts, target muscles, secondary muscles, and equipment in one view, enabling the user to interpret the movement without navigating between unrelated pages.

4.7.4 Administrative and Trainer Interfaces

Administrative pages use reusable data-table, pagination, status-badge, form-field, and confirmation-dialog components. User management presents account identity, role, and status with controlled actions. Exercise management presents lifecycle and editing controls. Trainer pages isolate profile and submission activities from administrative moderation. The visual separation supports both usability and the conceptual integrity of the role model.

4.7.5 Reusable UI Components

Table 4.8. Shared frontend components

Component	Purpose in the core demo
ConfirmDialog	Require explicit confirmation before destructive or lifecycle-changing actions
DataTable	Provide a reusable structure for administrative lists
EmptyState	Explain when no records match the current conditions
ErrorBoundary	Prevent an unexpected React component error from breaking the complete interface
ErrorState	Present request failure and retry behavior consistently
FormField	Standardize labels, inputs, and validation messages
LoadingState	Provide visible feedback during asynchronous operations
PageHeader	Maintain consistent page titles and contextual actions
Pagination	Navigate bounded list responses
StatusBadge	Communicate account or content lifecycle states

4.8 Principal End-to-End Workflows

The main FiFB workflows connect the React frontend, Express backend, and PostgreSQL database. The frontend receives user actions, while the backend validates requests, checks permissions, applies business rules, and accesses the database.

For authentication, the user submits an email and password through the login page. The backend verifies the account and returns an authenticated session. Protected requests are then checked against the current account status and role before the requested function is executed.

For exercise discovery, the frontend sends the keyword, selected filters, and pagination parameters to the exercise API. The backend retrieves active exercises together with their body-part, muscle, and equipment information, then returns the matching results to the exercise library.

For exercise contribution, a Trainer submits new content in a pending state. An Admin reviews the submission and may approve or reject it. Only active exercises are displayed to ordinary users, which helps maintain control over the shared exercise data.

For user management, an Admin can search for accounts and update their status. Locked or disabled accounts are prevented from accessing protected functions, while their existing data remain stored in the system.

Chapter 5 – Results and Evaluation

5.1 Results Achieved in GR1

The GR1 stage produced a working web-platform foundation that connects a React frontend, an Express backend, and a Supabase-hosted PostgreSQL database. The repository defines separate User, Trainer, and Admin roles; implements account authentication and protected routes; stores a normalized exercise dataset; provides user and exercise management interfaces; supports Trainer profile and submission functions; and enables exercise discovery through keyword and taxonomy filters.

The result should be interpreted as a foundation rather than as a complete commercial fitness community. The core identity, content, and authorization structures are present and can support later modules. Community interaction, comprehensive workout tracking, large-scale operational testing, and AI-assisted guidance require further refinement and evaluation during GR2.

5.2 Exercise Data Acquisition Result

The ExerciseDB import script was executed successfully against ExerciseDB V1 API Free Version, and the acquired exercise records were persisted to the project's Supabase database. The resulting dataset contains the structural fields required by the application, including name, GIF, instructions, body part, equipment, target muscle, and secondary muscles. The normalized taxonomy and junction tables make the data usable for filtering and administrative maintenance rather than preserving it only as an opaque external JSON response.

5.3 System Operation

When the system starts, the Express application initializes configuration and the database connection, registers middleware and domain routes, and exposes a health endpoint. The React application loads its route tree and authentication provider. Public exercise pages can request active content without an authenticated management session. When an account logs in, the frontend loads the current identity and renders navigation according to the stored role.

For an ordinary user, the principal operation is exercise lookup. The user selects search conditions, the backend queries the local database, and the interface renders a bounded result page. For a Trainer, the system adds profile and submission routes. For an Admin, the interface exposes user governance, exercise lifecycle management, and

taxonomy support. All management calls are checked by backend authorization, so direct API access does not bypass role restrictions.

5.4 Application Interfaces

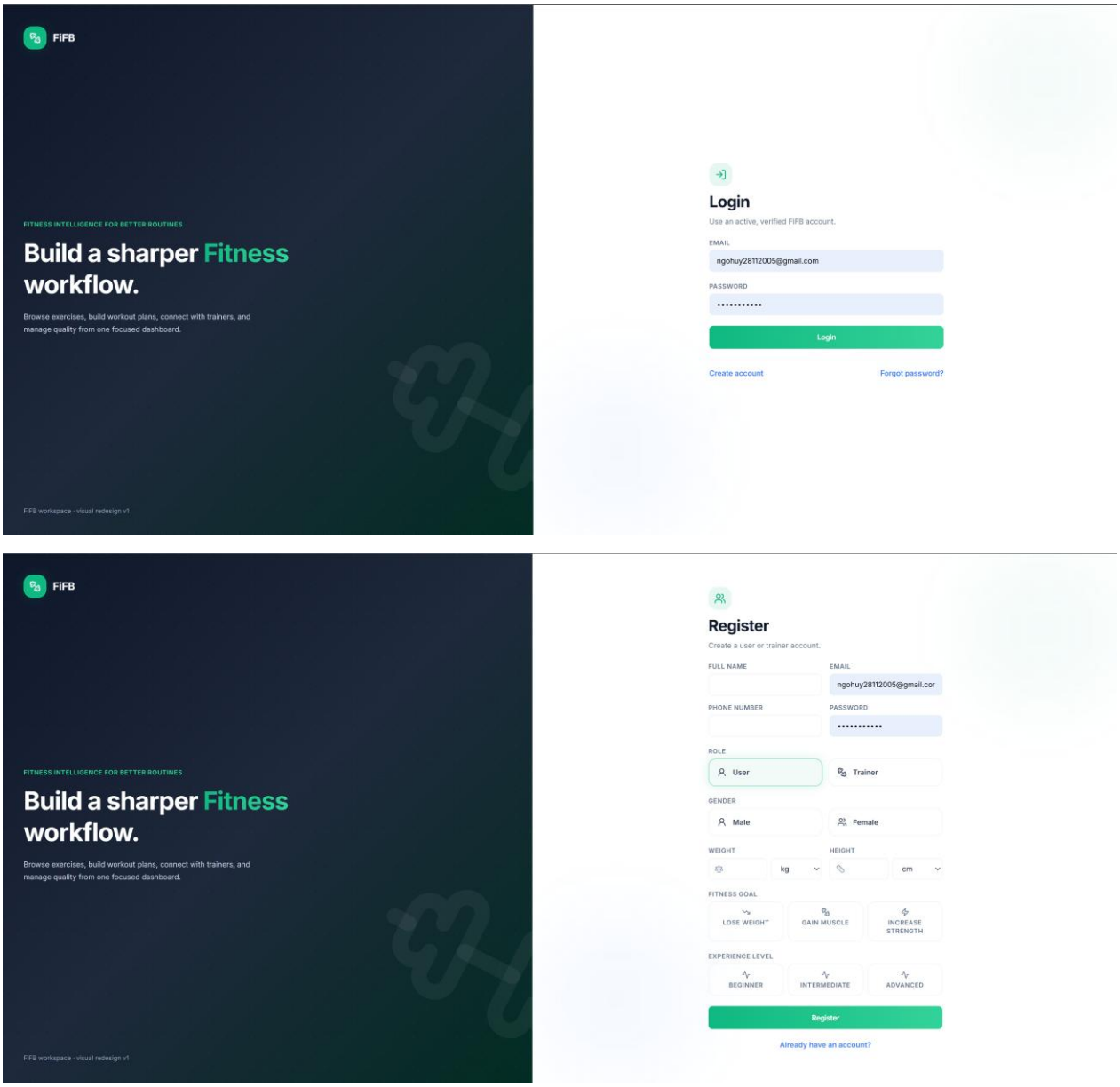


Figure 5.1. Login and registration interfaces

The authentication screens separate branding from the account form and provide direct transitions between login, registration, and recovery-related actions. The registration interface captures the information required to create an account and select an appropriate role. Form validation and backend error messages are expected to prevent incomplete or invalid submissions from being silently accepted.

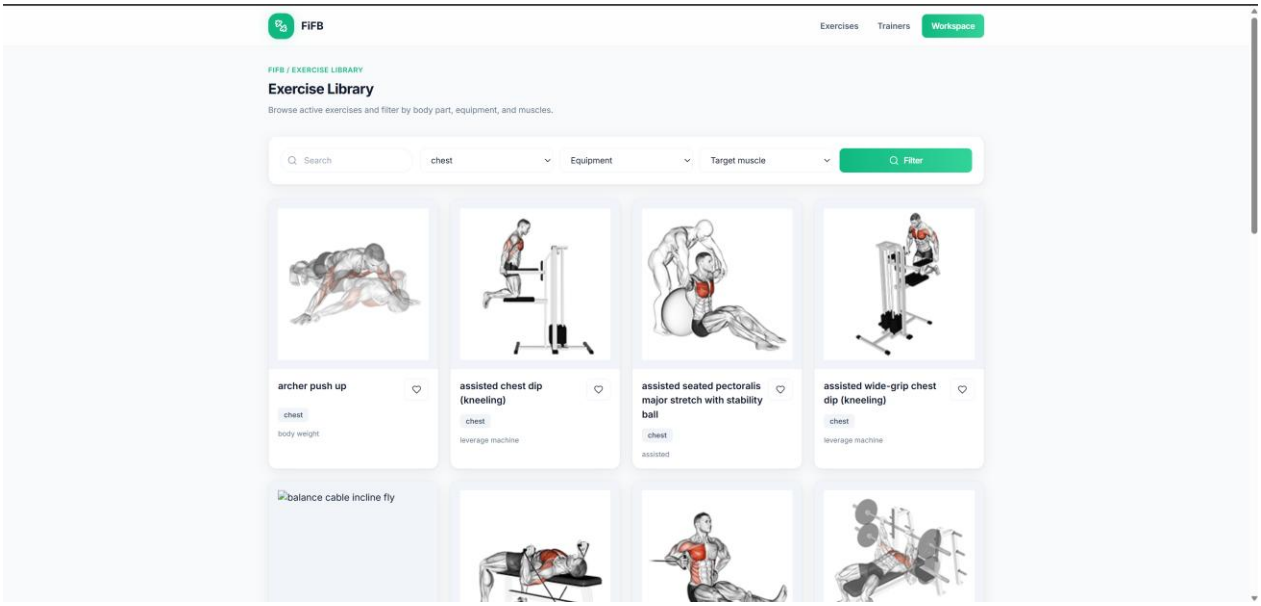


Figure 5.2. User exercise library with search and taxonomy filters

The exercise-library interface presents a search field and structured filters above a card-based result area. Each card provides an immediate visual reference and concise metadata, allowing the user to scan multiple options before opening a detail page. The layout reflects the market-survey conclusion that beginner discovery should remain direct and should not be hidden behind workout logging or business-management functions.

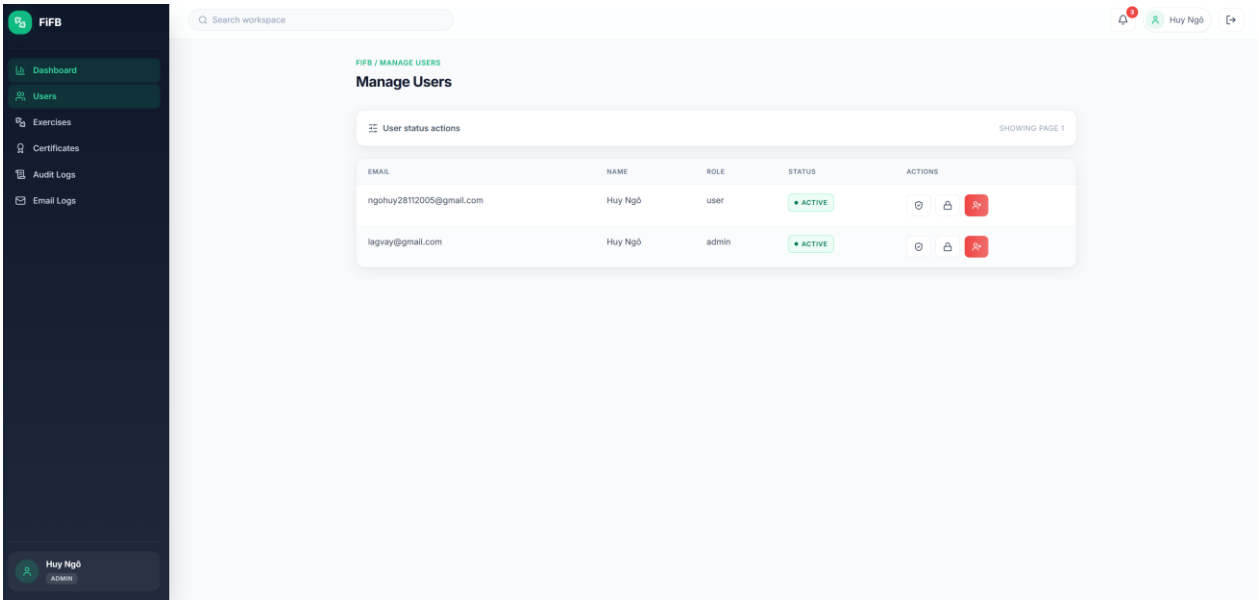


Figure 5.3. Admin user-management interface

The user-management screen places account records in a table and exposes status-related actions within the administrative layout. Search and filtering reduce the amount of information displayed at one time.

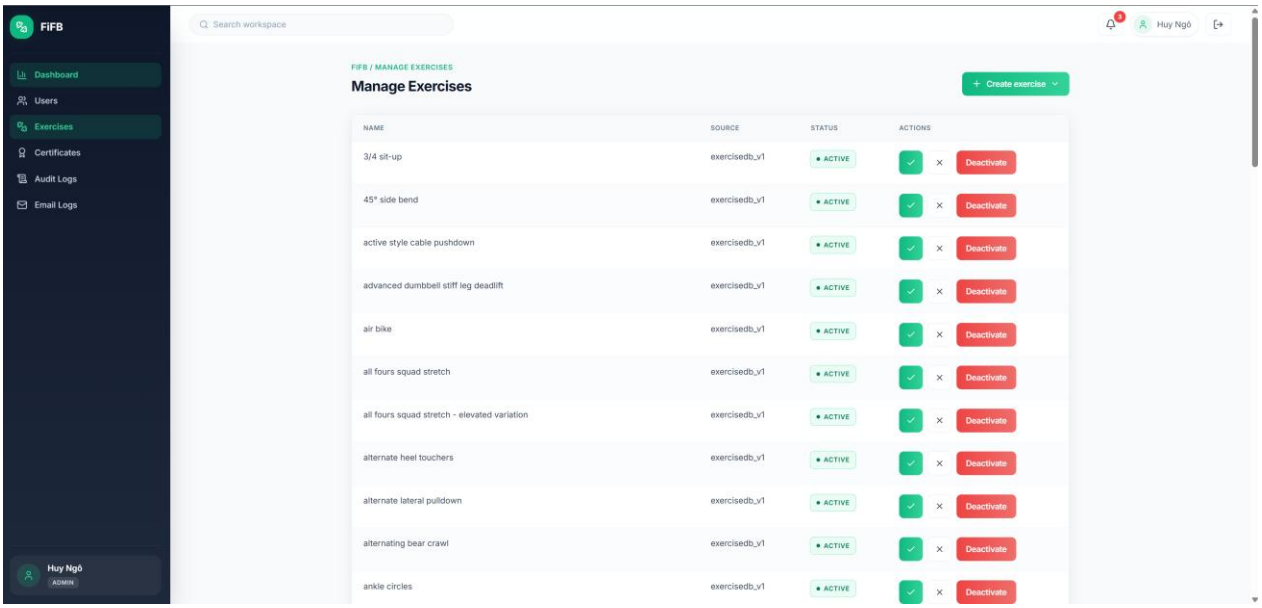


Figure 5.4. Admin exercise-management interface

The exercise-management screen displays content records and their lifecycle state, together with administrative actions. This interface supports the moderation model in which exercise data are governed rather than published without review.

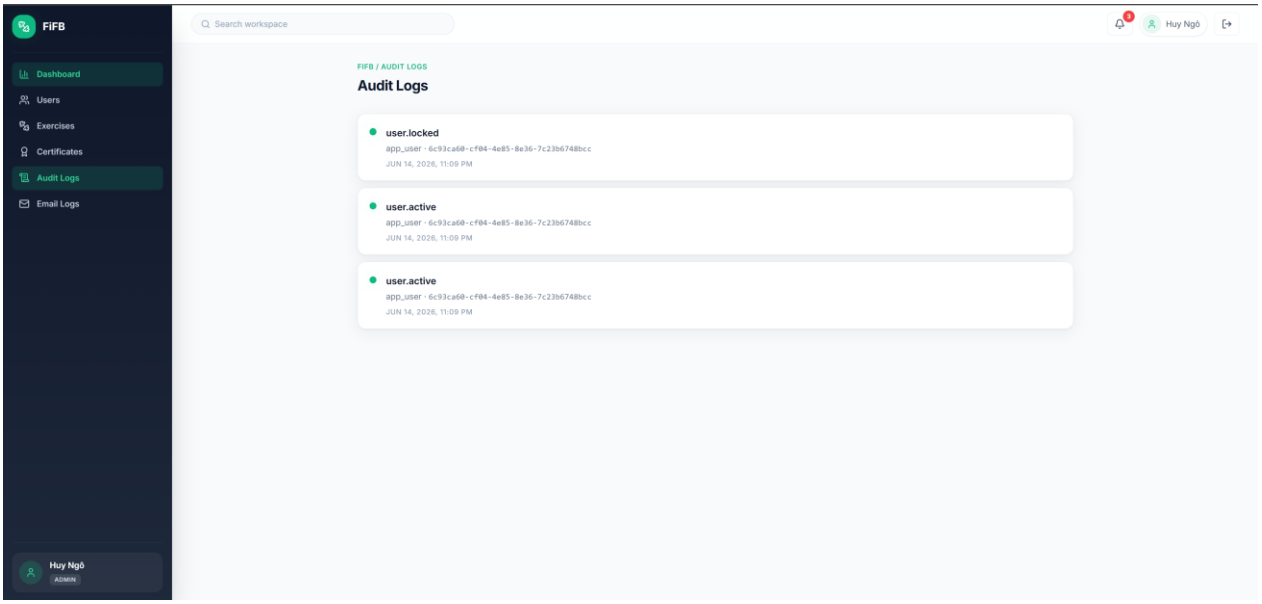


Figure 5.5. Administrative audit-log interface retained from the broader repository

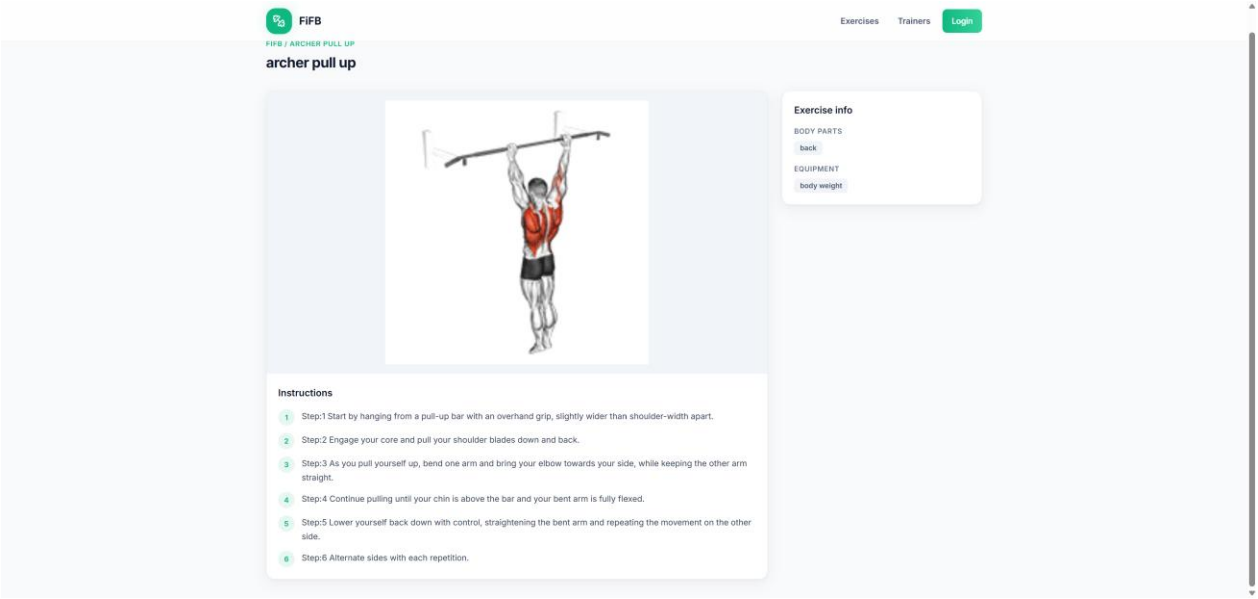


Figure 5.6. Exercise-detail page

5.5 Qualitative Evaluation of Software Quality

Usability is supported by role-specific navigation, visible search controls, card-based exercise presentation, reusable status indicators, and consistent form patterns. The current evidence is visual and implementation-oriented; no user study has yet measured task completion time, error rate, or perceived usability. Such a study would be appropriate after the interface and dataset stabilize.

Security is addressed through bcrypt password hashing, JWT verification, backend role checks, request validation, CORS, security headers, and parameterized SQL. These mechanisms provide a reasonable foundation, but production security would additionally require secret management, token-revocation policy, dependency monitoring, security testing, and operational logging.

Maintainability is one of the stronger aspects of the repository. Backend domain modules follow a repeated layered pattern, and frontend pages are separated from API and route infrastructure. The modular structure enables optional features to exist without changing the central user and exercise modules. The principal maintainability risk is scope growth: extension modules must continue to respect boundaries instead of introducing duplicated identity, authorization, or exercise logic.

Data integrity is supported by normalization, foreign keys, status constraints, upsert logic, and transactions. The ExerciseDB import design reduces duplication and synchronizes taxonomy relationships. Reliability is supported by centralized error handling and frontend feedback components. Performance has not been quantitatively

assessed, although pagination and indexed relational queries are appropriate design choices for the current scale.

5.6 Limitations

The first limitation is the absence of archived quantitative evaluation evidence. The report does not contain a complete Jest or Supertest execution log, code-coverage result, end-to-end test report, response-time benchmark, or load test. Consequently, the evaluation cannot state numerical reliability or performance. The next phase should automate the principal role scenarios and preserve build and test outputs as project artifacts.

The second limitation is data provenance and quality control. ExerciseDB supplies a useful initial dataset, but imported records may contain naming inconsistencies, incomplete instructions, media availability changes, or content that requires expert review. The Admin moderation model helps, but the system still requires explicit data-quality rules and a repeatable import audit process.

The third limitation is that the Trainer workflow remains foundational. Profile and exercise submission are sufficient to demonstrate role separation, but they do not yet establish a complete support relationship between trainers and users. Community interaction, reputation, reporting, moderation policy, and user safety require additional requirements analysis rather than only more interface pages.

5.7 Development Plan for GR2

Table 5.3. Proposed GR2 development plan

Priority	Planned work	Expected evaluation
Core verification	Add integration and end-to-end tests for login, role restrictions, user status, exercise lifecycle, Trainer submission, and search filters	Recorded test results, coverage, and reproducible execution
Data quality	Persist import logs, detect duplicates, review missing media or instructions, and define moderation criteria	Dataset statistics and quality report
Community refinement	Clarify trainer discovery, connections, reviews, reporting, and moderation without turning the product into gym-facility software	User journeys and role-specific acceptance tests
UI/UX	Improve responsive behavior, accessibility, content hierarchy, and Vietnamese localization	Usability testing with beginner users
AI-assisted support	Investigate personalized recommendation, conversational guidance, or movement-related assistance within clear safety boundaries	Baseline comparison, task-specific metrics, and human review
Operations	Strengthen deployment, monitoring, secret management, caching, and real-time reliability where required	Deployment checklist and operational test evidence

The AI work should begin with a narrowly defined task. For example, an exercise recommender could use user goals, available equipment, and experience level, while a conversational assistant could answer questions only from the verified FiFB knowledge base. Any posture or technique analysis would require an appropriate dataset, expert labeling, privacy controls, and safety evaluation. The GR2 contribution should therefore be measured by demonstrated user value and validated behavior rather than by the mere presence of an AI API.

References

[1] World Health Organization, “Physical activity,” Fact Sheet, 26 June 2024. https://www.who.int/news-room/fact-sheets/detail/physical-activity

[2] World Health Organization, Global levels of physical inactivity in adults: off track for 2030, 2024. https://www.who.int/publications/i/item/9789240096905

[3] JEFIT, “Workout Planner and Tracking Application.” https://www.jefit.com/

[4] Hevy, “Workout Tracker and Gym Logger.” https://www.hevyapp.com/

[5] ABC Trainerize, “Personal Training Software for Fitness Professionals.” https://www.trainerize.com/

[6] Nike, “Nike Training Club App.” https://www.nike.com/vn/ntc-app

[7] ExerciseDB, “EXERCISEDB V1 API – Free Version Documentation.” https://oss.exercisedb.dev/docs

[8] Meta Open Source, “React Documentation.” https://react.dev/

[9] Vite, “Vite Documentation.” https://vite.dev/

[10] React Router, “React Router Documentation.” https://reactrouter.com/

[11] Axios, “Axios Documentation.” https://axios-http.com/docs/intro

[12] Tailwind Labs, “Tailwind CSS Documentation.” https://tailwindcss.com/docs

[13] OpenJS Foundation, “Node.js Documentation.” https://nodejs.org/docs/latest/api/

[14] Express.js, “Express Web Framework Documentation.” https://expressjs.com/

[15] PostgreSQL Global Development Group, “PostgreSQL Documentation.” https://www.postgresql.org/docs/

[16] Supabase, “Database Documentation.” https://supabase.com/docs/guides/database

[17] IETF, RFC 9110: HTTP Semantics, 2022. https://www.rfc-editor.org/rfc/rfc9110

[18] IETF, RFC 7519: JSON Web Token (JWT), 2015. https://www.rfc-editor.org/rfc/rfc7519

[19] OWASP Foundation, “Password Storage Cheat Sheet.” https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[20] Zod, “TypeScript-first Schema Validation.” https://zod.dev/

[21] Helmet, “Helmet Documentation.” https://helmetjs.github.io/

[22] Jest, “Jest Documentation.” https://jestjs.io/docs/getting-started

[23] SuperAgent Team, “Supertest.” https://github.com/ladjs/supertest

[24] N. D. Huy, “Graduation-Research-FiFB,” GitHub repository, 2026. https://github.com/ngohuyhust/Graduation-Research-FiFB

[25] ISO, “ISO/IEC 25010 – Systems and software Quality Requirements and Evaluation.” https://www.iso.org/standard/78176.html

Appendices

Appendix A – Core API Documentation

The complete endpoint specification is maintained in Document/API.md in the project repository. For the focused GR1 report, the relevant sections are authentication and token handling, User and role permissions, Trainer profile and exercise submission, exercise discovery and lifecycle management, taxonomy endpoints, and administrative user-status operations.

Appendix B – Database Schema

The complete schema is maintained in Document/DatabaseSQL-v2.txt. The focused analysis in Chapter 4 uses app_users, trainer_profiles, exercises, body_parts, equipments, muscles, exercise_body_parts, exercise_equipments, exercise_muscles, and audit_logs. Extension tables remain in the schema for the broader community direction.

Appendix C – Focused Repository Structure

```
Graduation-Research-FiFB/
├── Document/                    API, database, problem statement, and diagrams
├── SystemDesign/                Broader architectural design artifacts
├── client/                      React + Vite frontend
│   └── src/                     API, components, contexts, hooks, layouts,
pages, routes
├── server/                      Node.js + Express backend
│   ├── scripts/                 Migration and ExerciseDB import scripts
│   ├── src/                     Configuration, database, middleware, modules,
routes
│   └── tests/                   Automated-test support
├── docker-compose.yml           Multi-service local environment
├── render.yaml                 Deployment blueprint
└── .github/workflows/           CI workflow definitions
```

The focused GR1 modules are auth, users, trainers, exercises, exerciseTaxonomy, and admin; all other modules remain GR2 extensions and are not counted as completed GR1 objectives.